Digital Image Processing — Assignment 1

Released 25th August · Due September 7th, 23:59

Report and code 50%, viva 50%. Your viva mark caps your report mark for each problem.

From scratch means NumPy array arithmetic. You may not call `scipy.ndimage`, `scipy.signal`, `cv2` or `skimage` for any operation a problem asks you to implement; use them to check your answers. `numpy.fft` is allowed.

Every number in your report must come from your machine. State your CPU, RAM and NumPy version.

Use the supplied photographs where a problem names a path; source your own only where a problem explicitly asks you to. Puzzle assets — the stego image, the transform pairs, the Q4 image sets, the exposure set, and the kernel bank — are supplied. Credit everything you add.

#	Topic	Marks
1	Convolution: naive, im2col, separability, low rank	30
2	Bit-plane slicing and robust steganography	20
3	Piecewise linear transforms	10
4	Histograms: equalisation, specification, local processing	25
5	Green screen keying and compositing	15

Problem 1 — Convolution (30)

Data. Use `images/p1/base_2048.png`, a 2048×2048 greyscale photograph with detail at every scale.

1.1 Correctness (6)

Implement 2-D convolution four ways: `conv2d_loops` (four nested Python loops), `conv2d_taps` (loop over taps, vectorised over the image), `conv2d_im2col` (patch matrix, one matrix–vector product), and `conv2d_fft`.

All four must agree. Use `mode='same'`, zero padding, and true convolution. Verify against `scipy.signal.convolve2d` and report $\max|\text{error}|$ for each.

Include one test with an asymmetric kernel showing your output differs from correlation.

Time `conv2d_loops` on a 128×128 crop at k=7 and extrapolate to 2048×2048, k=15. Report both, and say why you were not asked to run the full-size version.

1.2 Kernel rank (6)

Use the supplied bank at $k = 15$. It is defined, for general size k, with $\sigma = k/6$ and $r^2 = x^2 + y^2$ measured from the centre:

`box`	$k \times k$ of $1/k^2$. Unweighted local average.
`gaussian`	Outer product of a 1-D Gaussian with itself.
`sobel3`	$[1, 2, 1]^{\mathsf{T}}[-1, 0, 1]$, 3×3. Smooths across one axis while differencing along the other.
`log`	Laplacian of Gaussian, $\frac{r^2 - 2\sigma^2}{\sigma^4} \exp\left(\frac{-r^2}{2\sigma^2}\right)$. A blob and edge detector.
`log_dc_removed`	The same kernel with its mean subtracted, so its taps sum to zero.
`motion_0deg`	A single centre row of $1/k$, zero elsewhere. Horizontal linear motion blur.
`motion_45deg`	The main diagonal, I/k. An analogous sampled line blur at 45°.
`disk`	Uniform over $r \leq k/2$, normalised. Defocus.
`random`	I.i.d. Gaussian taps, L_1-normalised. A control with no structure.

For each, report the numerical rank and plot the normalised singular value spectrum. State the threshold you used to call a singular value zero, and why that choice matters.

(a) Derive why the raw LoG is exactly rank 2.

(b) For this sampled LoG, subtracting the mean raises the rank from 2 to 3. A constant matrix has rank 1, but a rank-1 perturbation does not always raise rank. Explain both the upper bound and why equality holds here.

(c) Motion blur at 0° is rank 1; at 45° it is full rank. The same type of physical operation has very different sampled-matrix rank. Explain.

1.3 Separability and low rank (10)

A rank-1 kernel factors as $K = \sigma_1 u_1 v_1^{\mathsf{T}}$, so the cost drops from $O(HWk^2)$ to $O(2HWk)$. Implement `conv2d_separable` for rank-1 kernels and `conv2d_lowrank(img, K, r)` summing r separable passes from the truncated SVD.

Plot PSNR against r for the disk, LoG, motion-45° and random kernels at k=21, on the top-left 512×512 crop of the supplied photograph, measured against the exact convolution.

Disk and motion-45° are both non-separable, yet three terms give high fidelity for the disk while the diagonal motion kernel needs all k terms for exact recovery. What distinguishes them, and how would you predict it from the singular values alone?

1.4 Runtime analysis (8)

Two log–log sweeps with fitted slopes: runtime against $k \in \{3, 7, 11, 15, 21, 31\}$ at 512^2, and against $N \in \{128, 256, 512, 1024, 2048\}$ at k=15. Report peak im2col patch-matrix memory at each N, derive the formula, and state what happens at N=2048, k=15.

You are graded on the discussion, not the plots.

(a) Theory says the tap loop scales as k^2. Your fitted slope will be less than 2. Where does the missing exponent go?

(b) im2col is standard in every deep learning framework. Measure whether it beats the tap loop on your machine for this task. Explain your result either way, state when im2col wins, and say whether those conditions hold here.

(c) Separable convolution should be $k/2$ times faster at k=15. Measure the speedup at 512^2 and at 2048^2. They differ. What changed?

Deliverables. Error table; rank table with spectra; PSNR-against-r curve; two log–log plots with slopes; memory table; written answers to 1.2 and 1.4.

Problem 2 — Bit-plane slicing and steganography (20)

Data. Use the two 8-bit greyscale covers `images/p2/cover_textured.png` and `images/p2/cover_smooth.png`. Both are 1024×1024. The supplied stego image is `images/p2/decode_me.png`; its clean cover is `images/p2/cover_textured.png`.

2.1 Decomposition (5)

Decompose the textured cover into 8 bit planes and display them. Rebuild from the top n planes and plot PSNR against n. The gain per plane is close to constant — measure it and explain why that value is not a coincidence.

Decompose a linear intensity ramp. Its low planes show a structure the natural image does not. Identify it, then re-run with Gray-coded intensities and explain the difference.

2.2 Hide and recover a payload (5)

The supplied `decode_me.png` carries a hidden payload in bit plane 0.

Read the least significant bit of every pixel in raster order — row by row from the top-left — to get one bit stream. It is laid out as:

bits 0–7	sync marker, `10101010`
bits 8–15	payload height, unsigned 8-bit, most significant bit first
bits 16–23	payload width, same encoding
bits 24 onwards	the payload, one bit per pixel, raster order

The payload is a binary image. Recover and display it. If your sync marker does not appear, your bit or pixel ordering is wrong — that is what it is for.

Report PSNR between the stego image and the supplied clean cover, say whether you could have detected the payload by eye, and produce a difference map of which pixels were changed by embedding. Note that all header/payload positions carry bits, including positions whose original LSB already equalled the embedded bit.

Then embed the same payload of at least 256×256 bits, at the same raster positions, in *both* covers. One is a materially worse hiding place. Show evidence, not an assertion; PSNR alone is not sufficient because equal numbers of one-level changes have nearly cover-independent PSNR.

2.3 Survive the channel (10)

Naive LSB embedding dies under the supplied rounded zero-mean Gaussian channel at $\sigma = 1$: its bit error rate is already close to 0.5, which is chance.

Design an embedding that carries **128 bits** and survives, subject to $\mathrm{PSNR}(\mathrm{cover}, \mathrm{stego}) \geq 40$ dB.

Survives means you recover all 128 bits with no errors at $\sigma = 5$, and separately after JPEG compression at quality 75, using the supplied channel functions unchanged and their default deterministic settings.

Two things to settle and state explicitly:

(a) **Does your decoder need the clean cover?** A decoder that has the original to compare against is solving an easier problem than one that does not. Either is acceptable. Say which you built, and what it would take to do without.

(b) **What are you trading?** 128 bits in a 1024^2 image is a very low rate, so the 40 dB floor is not the binding constraint — work out how much distortion budget it actually gives you, per modified pixel, and say what that buys.

Evaluate against additive Gaussian noise, $\sigma \in \{0, 1, 2, 5, 10, 20\}$, and JPEG at quality 75. Plot BER against σ for your scheme and naive LSB on the same axes; report the JPEG result for both and your stego PSNR.

The two channels break different candidate schemes. During design, if a candidate survives one and not the other, use that result to explain the mechanism and revise the design. The final submitted scheme must meet the zero-error requirement above for both $\sigma = 5$ noise and JPEG quality 75.

One paragraph justifying the design: where you sat on the capacity / robustness / imperceptibility triangle and what you gave up.

Deliverables. Eight plane images; PSNR-against-n plot; ramp and Gray-code comparison; recovered payload; difference map; textured-against-smooth evidence; BER curves; statement of blind or cover-assisted decoding; design paragraph.

Problem 3 — Piecewise linear transforms (10)

Data. The supplied pairs are A: `images/p3/field_in.png`→`field_out_A.png`, B: `images/p3/field_in.png`→`field_out_B.png`, C: `images/p3/nebula_in.png`→`nebula_out_C.png`, and D: `images/p3/field_skycrop_in.png`→ `field_skycrop_out_D.png`. They are 8-bit greyscale deep-space images with a large dark sky floor, faint mid-tones and small highlights.

You are given input/output image pairs. One pointwise piecewise-linear transform was applied to each. Recover it. That it is pointwise is the entire hint.

3.1 Recover two known transforms (4)

A is a levels operation: flatten the sky floor, stretch the faint nebulosity, roll off the highlights. **B** is grey-level slicing and is not monotonic.

For each, report breakpoints and per-segment slopes, overlay your fitted curve on the measured scatter, then re-apply your recovered transform to the input and report maximum and mean absolute error against the true output.

B is recoverable from the pair but not invertible from the output alone. Explain the difference in one sentence.

3.2 Choose the number of segments (3)

For the third pair you are not told how many segments the transform has. Select the count from the data.

A least-squares fit always improves as you add segments, so you need a complexity penalty. State what you used and why.

Below some residual you are no longer fitting the transform but the quantisation of the output lookup table, and a single true slope will decompose into alternating integer-valued segments. Demonstrate that your method does not do this.

3.3 Identifiability (3)

You can only observe $T[v]$ — the transform's output for input level v — at levels v that actually occur in the input image. If no pixel has value v, the data says nothing whatsoever about $T[v]$, and any transform agreeing everywhere else fits equally well. A parameter the data cannot distinguish is *unidentifiable*, and no method recovers it. Recognising that is the skill this part is testing.

The fourth pair is a crop containing almost nothing but sky, so only a narrow band of its input levels is well supported; isolated levels outside that band may occur in very few pixels. The true transform has **four** segments.

(a) Plot the histogram of the input crop and state the range of input levels that are actually sampled. Choose and justify a threshold for how many pixels a level needs before you will trust $T[v]$ there.

(b) Recover what the data supports. Report the segments you can determine, and give the intensity range over which your answer is unconstrained.

(c) How many segments did you report, and why is that not four? A correct answer reports fewer and justifies the omission.

Deliverables. Scatter with fitted overlay per case; breakpoint and slope table; re-application error; input histogram for 3.3 with your support threshold marked; explicit statement of which intensity ranges the input fails to constrain.

Problem 4 — Histograms (25)

Data. Use the supplied photographs in `images/p4/`: 4.1 uses `equalization/input.png`; 4.2(a) uses `matching/source.png` and `matching/reference.png`; 4.2(b) uses `specification/source.png`; 4.2(c) uses `colour_source.png`; 4.3 uses `local_regions.png`. The five files `exposure/ev0.png`–`ev4.png` belong only to 4.4.

4.1 Global equalisation (5)

On `equalization/input.png`, implement CDF-based equalisation from scratch. For consistent marking, use $T(v) = \text{round}(255F(v))$, where F is the normalised inclusive CDF; do not apply a CDF_{min} rescaling. Show image and histogram before and after, plus both CDFs.

(a) Your equalised histogram is not flat. Prove that it cannot be.

(b) Show that equalisation is exactly idempotent. Prove it, do not observe it.

(c) Find or construct an image on which equalisation does essentially nothing, and explain what property of its histogram causes that.

4.2 Histogram matching and specification (6)

Implement inverse-CDF composition for an arbitrary target histogram. Use the supplied photographs as follows.

(a) **Image matching.** Match `matching/source.png` to the histogram of `matching/reference.png`. Show source, reference and result with their histograms and CDFs.

(b) **Histogram specification.** Apply a uniform target and a Gaussian target ($\mu = 128$, $\sigma = 35$) to `specification/source.png`. These targets are prescribed 256-bin distributions, not target pictures. Explain how uniform specification differs from plain equalisation, if at all.

(c) **Colour.** On `colour_source.png`, equalise each RGB channel independently and equalise luminance only. Measure mean hue shift for each and show both results. State your colour space and circular hue-distance formula, and exclude near-achromatic pixels using a stated saturation threshold because their hue is undefined. Explain why the per-channel result is wrong in terms of what equalisation assumes about its input.

For all three matching/specification targets, report Wasserstein-1 distance between achieved and target histograms before and after; do not score by eye.

4.3 Local histogram processing (10)

Use `local_regions.png` for every part below. Convert it once to 8-bit BT.601 luminance, $Y = \text{round}(0.299R + 0.587G + 0.114B)$, and use that same array throughout. A "tile count" t below means a $t \times t$ grid.

(a) Plot histograms of at least five distinct local patches and show that no single global mapping serves all of them.

(b) Implement plain tiled AHE: independent equalisation per tile, no clipping, no interpolation. It fails in two specific ways. Identify and explain both.

(c) Implement CLAHE: clip each tile histogram at a multiple of its mean bin height, redistribute the excess, and bilinearly interpolate between the four surrounding tile mappings at every pixel.

(d) Ablate clip limit over $\{1, 2, 3, 10, \infty\}$ and tile count over $\{2, 4, 8, 16, 64\}$ as an image grid. Describe both degenerate ends of each.

(e) Implement local histogram *specification* — matching each region to a target histogram rather than a uniform one. Show a case where it beats CLAHE and say what extra information you had to supply.

4.4 Automatic correction (4)

Given `exposure/ev0.png` through `ev4.png`, five exposures of one scene, correct all five to a consistent appearance with no per-image parameters. Your function takes an image and returns an image; anything it needs it estimates from the image.

Compute W1 on normalised 8-bit BT.601-luminance histograms. Report the mean and maximum of the ten pairwise distances before and after correction. Also report the output luma standard deviations and 1st–99th percentile ranges; a constant or near-constant output is invalid even though it makes every pairwise distance zero. State your estimator and give a concrete failure case.

Deliverables. Before/after images with histograms and CDFs; proofs for 4.1(a) and (b); W1 table for all three targets; per-channel against luminance with hue-shift numbers; five patch histograms; AHE failure images; clip and tile ablation grids; local specification comparison; corrected exposure set with score.

Problem 5 — Green screen compositing (15)

Data. You source the footage. 720p or better, a subject on a green backing, and it must contain hair, motion blur or translucent material — without partially transparent pixels a hard threshold matches a proper matte and 5.2 has no content. Avoid AI-generated or animated clips, whose soft edges may not represent real optical mixtures, and anything watermarked. Extract 48 frames. You also need a background plate at least 2.5× your frame width so it can pan. Say which clip you chose and why it meets the requirement.

5.1 Naive key (3)

Threshold on green dominance in raw RGB, then key on chroma alone in YCbCr or HSV. Identify a specific pixel where the RGB version fails and explain what property of RGB causes it. If your backing is evenly lit the failure may be mild — say so, and construct the case where it would not be.

5.2 Soft matte and spill suppression (7)

(a) Before choosing any threshold, plot chroma distance from the key colour separately for pixels you believe are pure backing, partially transparent, and pure foreground. State whether the three populations separate in your footage, and what follows if they do not.

(b) Produce continuous alpha from chroma distance and composite as $I = \alpha F + (1-\alpha)B$. Report the fraction of pixels with fractional alpha.

(c) Suppress the spill. Your correction should scale with $(1-\alpha)$; explain why. Measure residual green excess over fractional-alpha pixels only, and say why that restriction matters.

(d) You have no ground-truth alpha. Compare your naive and soft mattes against each other: mean absolute difference of the alpha maps, and separately of their gradient fields. The two disagree sharply. Explain why, and which of the two predicts what you see in the composite.

5.3 Video, efficiently (5)

Composite your 48 frames against a panning crop of the background plate and encode to mp4. Vectorise it. Report frames per second and name your bottleneck — measure it.

Per-frame independent keying flickers. Raw frame-to-frame alpha change also contains genuine subject motion, so measure flicker in a stated static region or after motion-aligning consecutive mattes. Apply a temporal filter, and show both that this flicker measure drops and that your

smoothing does not smear or lag the moving edge.

Deliverables. Clip source and justification; RGB failure image with the identified pixel; chroma distance distributions; alpha matte; composites with and without spill suppression; the two matte-difference numbers; mp4; frames per second with named bottleneck; flicker before and after.